LLaMA: Open and Efficient Foundation Language Models

Hugo Touvron,* Thibaut Lavril,* Gautier Izacard,* Xavier Martinet
Marie-Anne Lachaux, Timothee Lacroix, Baptiste Rozière, Naman Goyal
Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin
Edouard Grave,* Guillaume Lample*

Meta AI

arXiv:2302.13971v1 [cs.CL] 27 Feb 2023

Abstract

We introduce LLaMA, a collection of foundation language models ranging from 7B to 65B parameters. We train our models on trillions of tokens, and show that it is possible to train state-of-the-art models using publicly available datasets exclusively, without resorting to proprietary and inaccessible datasets. In particular, LLaMA-13B outperforms GPT-3 (175B) on most benchmarks, and LLaMA-65B is competitive with the best models, Chinchilla-70B and PaLM-540B. We release all our models to the research community[1].

1 Introduction

Large Languages Models (LLMs) trained on massive corpora of texts have shown their ability to perform new tasks from textual instructions or from a few examples (Brown et al., 2020). These few-shot properties first appeared when scaling models to a sufficient size (Kaplan et al., 2020), resulting in a line of work that focuses on further scaling these models (Chowdhery et al., 2022; Rae et al., 2021). These efforts are based on the assumption that more parameters will lead to better performance. However, recent work from Hoffmann et al. (2022) shows that, for a given compute budget, the best performances are not achieved by the largest models, but by smaller models trained on more data.

The objective of the scaling laws from Hoffmann et al. (2022) is to determine how to best scale the dataset and model sizes for a particular *training* compute budget. However, this objective disregards the *inference* budget, which becomes critical when serving a language model at scale. In this context, given a target level of performance, the preferred model is not the fastest to train but the fastest at inference, and although it may be cheaper to train a large model to reach a certain level of performance, a smaller one trained longer will ultimately be cheaper at inference. For instance, although Hoffmann et al. (2022) recommends training a 10B model on 200B tokens, we find that the performance of a 7B model continues to improve even after 1T tokens.

The focus of this work is to train a series of language models that achieve the best possible performance at various inference budgets, by training on more tokens than what is typically used. The resulting models, called *LLaMA*, ranges from 7B to 65B parameters with competitive performance compared to the best existing LLMs. For instance, LLaMA-13B outperforms GPT-3 on most benchmarks, despite being 10× smaller. We believe that this model will help democratize the access and study of LLMs, since it can be run on a single GPU. At the higher-end of the scale, our 65B-parameter model is also competitive with the best large language models such as Chinchilla or PaLM-540B.

Unlike Chinchilla, PaLM, or GPT-3, we only use publicly available data, making our work compatible with open-sourcing, while most existing models rely on data which is either not publicly available or undocumented (e.g. "Books – 2TB" or "Social media conversations"). There exist some exceptions, notably OPT (Zhang et al., 2022), GPT-NeoX (Black et al., 2022), BLOOM (Scao et al., 2022) and GLM (Zeng et al., 2022), but none that are competitive with PaLM-62B or Chinchilla.

In the rest of this paper, we present an overview of the modifications we made to the transformer architecture (Vaswani et al., 2017), as well as our training method. We then report the performance of our models and compare with others LLMs on a set of standard benchmarks. Finally, we expose some of the biases and toxicity encoded in our models, using some of the most recent benchmarks from the responsible AI community.

* Equal contribution. Correspondence: {htouvron, thibautlav,gizacard,egrave,glample}@meta.com

[1] https://github.com/facebookresearch/llama

2 Approach

Our training approach is similar to the methods described in previous work (Brown et al., 2020; Chowdhery et al., 2022), and is inspired by the Chinchilla scaling laws (Hoffmann et al., 2022). We train large transformers on a large quantity of textual data using a standard optimizer.

2.1 Pre-training Data

Our training dataset is a mixture of several sources, reported in Table 1, that cover a diverse set of domains. For the most part, we reuse data sources that have been leveraged to train other LLMs, with the restriction of only using data that is publicly available, and compatible with open sourcing. This leads to the following mixture of data and the percentage they represent in the training set:

English CommonCrawl [67%]. We preprocess five CommonCrawl dumps, ranging from 2017 to 2020, with the CCNet pipeline (Wenzek et al., 2020). This process deduplicates the data at the line level, performs language identification with a fastText linear classifier to remove non-English pages and filters low quality content with an n-gram language model. In addition, we trained a linear model to classify pages used as references in Wikipedia *v.s.* randomly sampled pages, and discarded pages not classified as references.

C4 [15%]. During exploratory experiments, we observed that using diverse pre-processed CommonCrawl datasets improves performance. We thus included the publicly available C4 dataset (Raffel et al., 2020) in our data. The preprocessing of C4 also contains deduplication and language identification steps: the main difference with CCNet is the quality filtering, which mostly relies on heuristics such as presence of punctuation marks or the number of words and sentences in a webpage.

Github [4.5%]. We use the public GitHub dataset available on Google BigQuery. We only kept projects that are distributed under the Apache, BSD and MIT licenses. Additionally, we filtered low quality files with heuristics based on the line length or proportion of alphanumeric characters, and removed boilerplate, such as headers, with regular expressions. Finally, we deduplicate the resulting dataset at the file level, with exact matches.

Wikipedia [4.5%]. We add Wikipedia dumps from the June-August 2022 period, covering 20 languages, which use either the Latin or Cyrillic scripts: `bg`, `ca`, `cs`, `da`, `de`, `en`, `es`, `fr`, `hr`, `hu`, `it`, `nl`, `pl`, `pt`, `ro`, `ru`, `sl`, `sr`, `sv`, `uk`. We process the data to remove hyperlinks, comments and other formatting boilerplate.

Dataset	Sampling prop.	Epochs	Disk size
CommonCrawl	67.0%	1.10	3.3 TB
C4	15.0%	1.06	783 GB
Github	4.5%	0.64	328 GB
Wikipedia	4.5%	2.45	83 GB
Books	4.5%	2.23	85 GB
ArXiv	2.5%	1.06	92 GB
StackExchange	2.0%	1.03	78 GB

Table 1: **Pre-training data.** Data mixtures used for pre-training, for each subset we list the sampling proportion, number of epochs performed on the subset when training on 1.4T tokens, and disk size. The pre-training runs on 1T tokens have the same sampling proportion.

Gutenberg and Books3 [4.5%]. We include two book corpora in our training dataset: the Gutenberg Project, which contains books that are in the public domain, and the Books3 section of ThePile (Gao et al., 2020), a publicly available dataset for training large language models. We perform deduplication at the book level, removing books with more than 90% content overlap.

ArXiv [2.5%]. We process arXiv Latex files to add scientific data to our dataset. Following Lewkowycz et al. (2022), we removed everything before the first section, as well as the bibliography. We also removed the comments from the .tex files, and inline-expanded definitions and macros written by users to increase consistency across papers.

Stack Exchange [2%]. We include a dump of Stack Exchange, a website of high quality questions and answers that covers a diverse set of domains, ranging from computer science to chemistry. We kept the data from the 28 largest websites, removed the HTML tags from text and sorted the answers by score (from highest to lowest).

Tokenizer. We tokenize the data with the byte-pair encoding (BPE) algorithm (Sennrich et al., 2015), using the implementation from SentencePiece (Kudo and Richardson, 2018). Notably, we split all numbers into individual digits, and fallback to bytes to decompose unknown UTF-8 characters.

params	dimension	n heads	n layers	learning rate	batch size	n tokens
6.7B	4096	32	32	$3.0e^{-4}$	4M	1.0T
13.0B	5120	40	40	$3.0e^{-4}$	4M	1.0T
32.5B	6656	52	60	$1.5e^{-4}$	4M	1.4T
65.2B	8192	64	80	$1.5e^{-4}$	4M	1.4T

Table 2: **Model sizes, architectures, and optimization hyper-parameters.**

Overall, our entire training dataset contains roughly 1.4T tokens after tokenization. For most of our training data, each token is used only once during training, with the exception of the Wikipedia and Books domains, over which we perform approximately two epochs.

2.2 Architecture

Following recent work on large language models, our network is based on the transformer architecture (Vaswani et al., 2017). We leverage various improvements that were subsequently proposed, and used in different models such as PaLM. Here are the main difference with the original architecture, and where we were found the inspiration for this change (in bracket):

Pre-normalization [GPT3]. To improve the training stability, we normalize the input of each transformer sub-layer, instead of normalizing the output. We use the RMSNorm normalizing function, introduced by Zhang and Sennrich (2019).

SwiGLU activation function [PaLM]. We replace the ReLU non-linearity by the SwiGLU activation function, introduced by Shazeer (2020) to improve the performance. We use a dimension of $\frac{2}{3}4d$ instead of $4d$ as in PaLM.

Rotary Embeddings [GPTNeo]. We remove the absolute positional embeddings, and instead, add rotary positional embeddings (RoPE), introduced by Su et al. (2021), at each layer of the network.

The details of the hyper-parameters for our different models are given in Table 2.

2.3 Optimizer

Our models are trained using the AdamW optimizer (Loshchilov and Hutter, 2017), with the following hyper-parameters: $\beta_1 = 0.9, \beta_2 = 0.95$. We use a cosine learning rate schedule, such that the final learning rate is equal to 10% of the maximal learning rate. We use a weight decay of 0.1 and gradient clipping of 1.0. We use $2,000$ warmup steps, and vary the learning rate and batch size with the size of the model (see Table 2 for details).

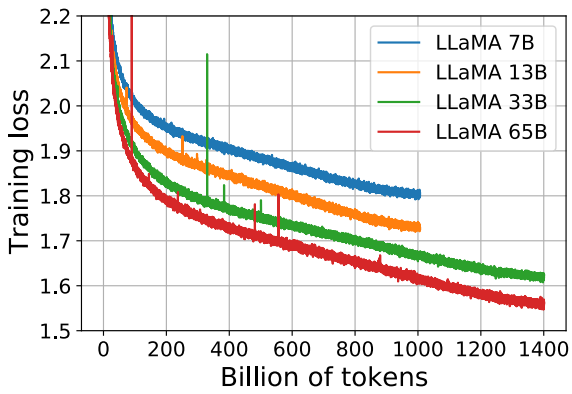

Figure 1: **Training loss over train tokens for the 7B, 13B, 33B, and 65 models.** LLaMA-33B and LLaMA-65B were trained on 1.4T tokens. The smaller models were trained on 1.0T tokens. All models are trained with a batch size of 4M tokens.

2.4 Efficient implementation

We make several optimizations to improve the training speed of our models. First, we use an efficient implementation of the causal multi-head attention to reduce memory usage and runtime. This implementation, available in the `xformers` library,[2] is inspired by Rabe and Staats (2021) and uses the backward from Dao et al. (2022). This is achieved by not storing the attention weights and not computing the key/query scores that are masked due to the causal nature of the language modeling task.

To further improve training efficiency, we reduced the amount of activations that are recomputed during the backward pass with checkpointing. More precisely, we save the activations that are expensive to compute, such as the outputs of linear layers. This is achieved by manually implementing the backward function for the transformer layers, instead of relying on the PyTorch autograd. To fully benefit from this optimization, we need to

[2] https://github.com/facebookresearch/xformers

		BoolQ	PIQA	SIQA	HellaSwag	WinoGrande	ARC-e	ARC-c	OBQA
GPT-3	175B	60.5	81.0	-	78.9	70.2	68.8	51.4	57.6
Gopher	280B	79.3	81.8	50.6	79.2	70.1	-	-	-
Chinchilla	70B	83.7	81.8	51.3	80.8	74.9	-	-	-
PaLM	62B	84.8	80.5	-	79.7	77.0	75.2	52.5	50.4
PaLM-cont	62B	83.9	81.4	-	80.6	77.0	-	-	-
PaLM	540B	**88.0**	82.3	-	83.4	**81.1**	76.6	53.0	53.4
LLaMA	7B	76.5	79.8	48.9	76.1	70.1	72.8	47.6	57.2
	13B	78.1	80.1	50.4	79.2	73.0	74.8	52.7	56.4
	33B	83.1	82.3	50.4	82.8	76.0	**80.0**	**57.8**	58.6
	65B	85.3	**82.8**	**52.3**	**84.2**	77.0	78.9	56.0	**60.2**

Table 3: **Zero-shot performance on Common Sense Reasoning tasks.**

reduce the memory usage of the model by using model and sequence parallelism, as described by Korthikanti et al. (2022). Moreover, we also overlap the computation of activations and the communication between GPUs over the network (due to `all_reduce` operations) as much as possible.

When training a 65B-parameter model, our code processes around 380 tokens/sec/GPU on 2048 A100 GPU with 80GB of RAM. This means that training over our dataset containing 1.4T tokens takes approximately 21 days.

3 Main results

Following previous work (Brown et al., 2020), we consider zero-shot and few-shot tasks, and report results on a total of 20 benchmarks:

- **Zero-shot.** We provide a textual description of the task and a test example. The model either provides an answer using open-ended generation, or ranks the proposed answers.
- **Few-shot.** We provide a few examples of the task (between 1 and 64) and a test example. The model takes this text as input and generates the answer or ranks different options.

We compare LLaMA with other foundation models, namely the non-publicly available language models GPT-3 (Brown et al., 2020), Gopher (Rae et al., 2021), Chinchilla (Hoffmann et al., 2022) and PaLM (Chowdhery et al., 2022), as well as the open-sourced OPT models (Zhang et al., 2022), GPT-J (Wang and Komatsuzaki, 2021), and GPT-Neo (Black et al., 2022). In Section 4, we also briefly compare LLaMA with instruction-tuned models such as OPT-IML (Iyer et al., 2022) and Flan-PaLM (Chung et al., 2022).

We evaluate LLaMA on free-form generation tasks and multiple choice tasks. In the multiple choice tasks, the objective is to select the most appropriate completion among a set of given options, based on a provided context. We select the completion with the highest likelihood given the provided context. We follow Gao et al. (2021) and use the likelihood normalized by the number of characters in the completion, except for certain datasets (OpenBookQA, BoolQ), for which we follow Brown et al. (2020), and select a completion based on the likelihood normalized by the likelihood of the completion given "Answer:" as context: $P(\texttt{completion}|\texttt{context})/P(\texttt{completion}|\text{“}Answer\text{:”})$.

		0-shot	1-shot	5-shot	64-shot
GPT-3	175B	14.6	23.0	-	29.9
Gopher	280B	10.1	-	24.5	28.2
Chinchilla	70B	16.6	-	31.5	35.5
PaLM	8B	8.4	10.6	-	14.6
	62B	18.1	26.5	-	27.6
	540B	21.2	29.3	-	39.6
LLaMA	7B	16.8	18.7	22.0	26.1
	13B	20.1	23.4	28.1	31.9
	33B	**24.9**	28.3	32.9	36.0
	65B	23.8	**31.0**	**35.0**	**39.9**

Table 4: **NaturalQuestions.** Exact match performance.

3.1 Common Sense Reasoning

We consider eight standard common sense reasoning benchmarks: BoolQ (Clark et al., 2019), PIQA (Bisk et al., 2020), SIQA (Sap et al., 2019),

HellaSwag (Zellers et al., 2019), WinoGrande (Sakaguchi et al., 2021), ARC easy and challenge (Clark et al., 2018) and OpenBookQA (Mihaylov et al., 2018). These datasets include Cloze and Winograd style tasks, as well as multiple choice question answering. We evaluate in the zero-shot setting as done in the language modeling community.

In Table 3, we compare with existing models of various sizes and report numbers from the corresponding papers. First, LLaMA-65B outperforms Chinchilla-70B on all reported benchmarks but BoolQ. Similarly, this model surpasses PaLM-540B everywhere but on BoolQ and WinoGrande. LLaMA-13B model also outperforms GPT-3 on most benchmarks despite being 10× smaller.

3.2 Closed-book Question Answering

We compare LLaMA to existing large language models on two closed-book question answering benchmarks: Natural Questions (Kwiatkowski et al., 2019) and TriviaQA (Joshi et al., 2017). For both benchmarks, we report exact match performance in a closed book setting, i.e., where the models do not have access to documents that contain evidence to answer the question. In Table 4, we report performance on NaturalQuestions, and in Table 5, we report on TriviaQA. On both benchmarks, LLaMA-65B achieve state-of-the-arts performance in the zero-shot and few-shot settings. More importantly, the LLaMA-13B is also competitive on these benchmarks with GPT-3 and Chinchilla, despite being 5-10× smaller. This model runs on a single V100 GPU during inference.

		0-shot	1-shot	5-shot	64-shot
Gopher	280B	43.5	-	57.0	57.2
Chinchilla	70B	55.4	-	64.1	64.6
LLaMA	7B	50.0	53.4	56.3	57.6
	13B	56.6	60.5	63.1	64.0
	33B	65.1	67.9	69.9	70.4
	65B	**68.2**	**71.6**	**72.6**	**73.0**

Table 5: **TriviaQA.** Zero-shot and few-shot exact match performance on the filtered dev set.

3.3 Reading Comprehension

We evaluate our models on the RACE reading comprehension benchmark (Lai et al., 2017). This dataset was collected from English reading comprehension exams designed for middle and high school Chinese students. We follow the evaluation setup from Brown et al. (2020) and report results in Table 6. On these benchmarks, LLaMA-65B is competitive with PaLM-540B, and, LLaMA-13B outperforms GPT-3 by a few percents.

		RACE-middle	RACE-high
GPT-3	175B	58.4	45.5
PaLM	8B	57.9	42.3
	62B	64.3	47.5
	540B	**68.1**	49.1
LLaMA	7B	61.1	46.9
	13B	61.6	47.2
	33B	64.1	48.3
	65B	67.9	**51.6**

Table 6: **Reading Comprehension.** Zero-shot accuracy.

3.4 Mathematical reasoning

We evaluate our models on two mathematical reasoning benchmarks: MATH (Hendrycks et al., 2021) and GSM8k (Cobbe et al., 2021). MATH is a dataset of 12K middle school and high school mathematics problems written in LaTeX. GSM8k is a set of middle school mathematical problems. In Table 7, we compare with PaLM and Minerva (Lewkowycz et al., 2022). Minerva is a series of PaLM models finetuned on 38.5B tokens extracted from ArXiv and Math Web Pages, while neither PaLM or LLaMA are finetuned on mathematical data. The numbers for PaLM and Minerva are taken from Lewkowycz et al. (2022), and we compare with and without `maj1@k`. `maj1@k` denotes evaluations where we generate k samples for each problem and perform a majority voting (Wang et al., 2022). On GSM8k, we observe that LLaMA-65B outperforms Minerva-62B, although it has not been fine-tuned on mathematical data.

3.5 Code generation

We evaluate the ability of our models to write code from a natural language description on two benchmarks: HumanEval (Chen et al., 2021) and MBPP (Austin et al., 2021). For both tasks, the model receives a description of the program in a few sentences, as well as a few input-output examples. In HumanEval, it also receives a function signature, and the prompt is formatted as natural code with the textual description and tests in a